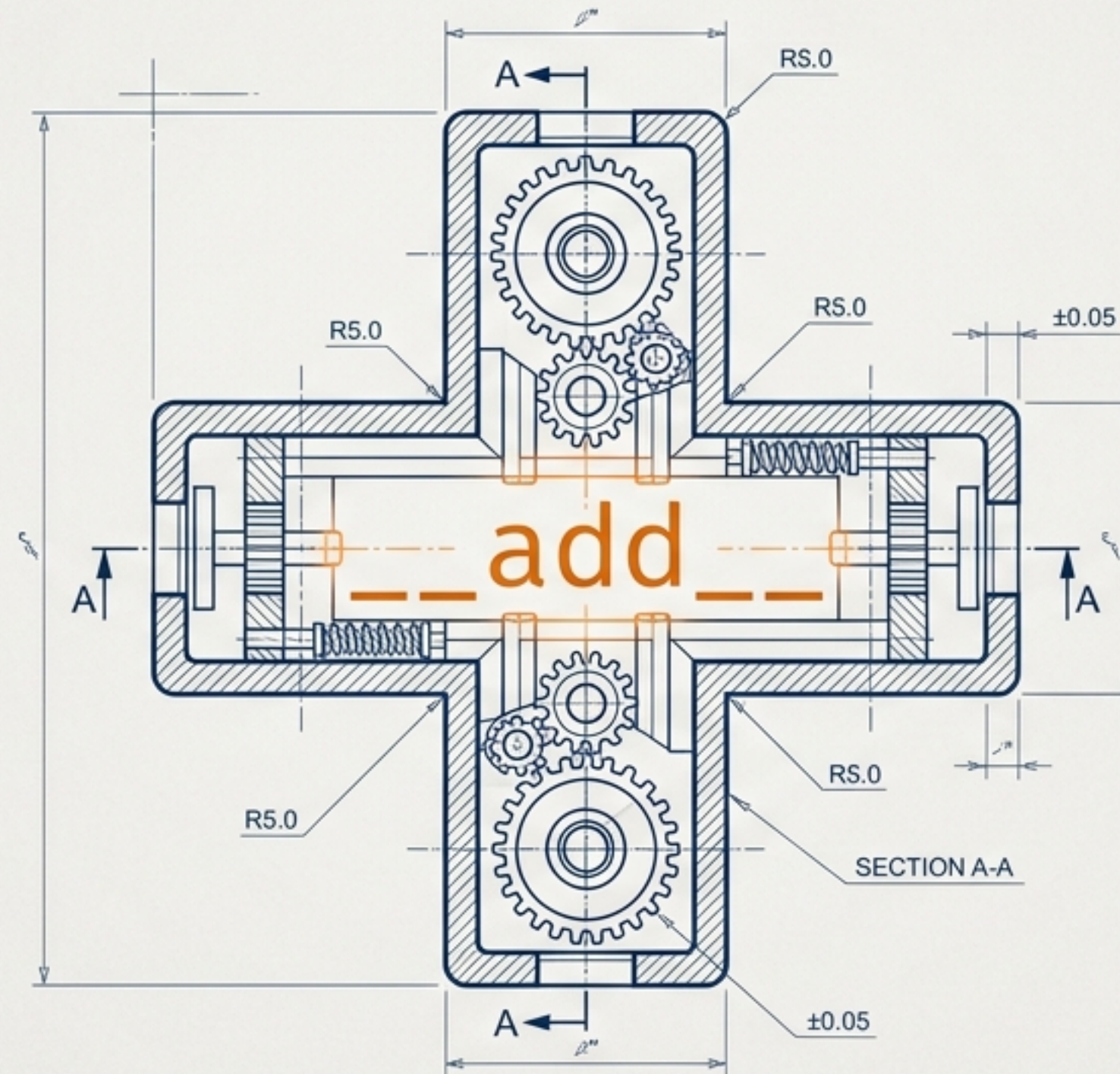
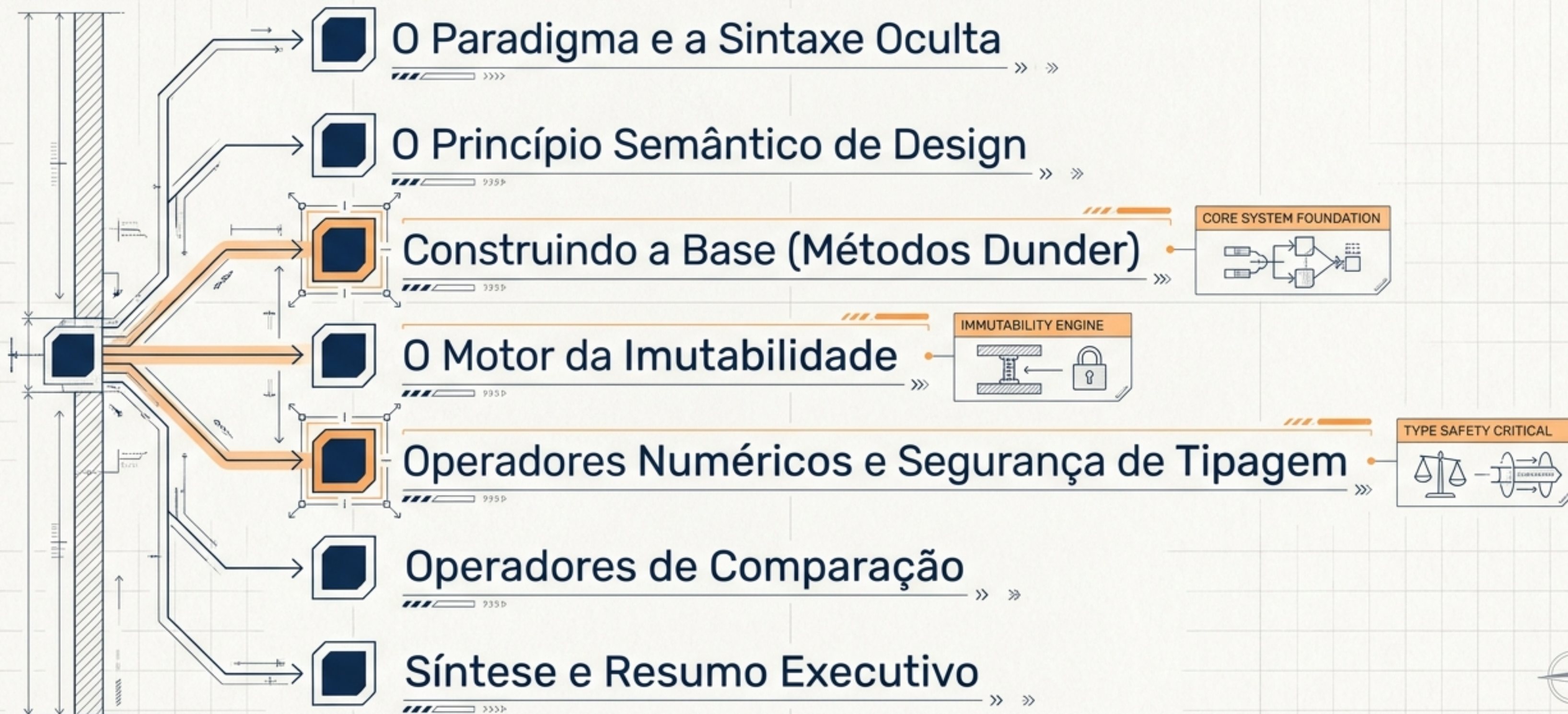


Sobrecarga de Operadores em Python

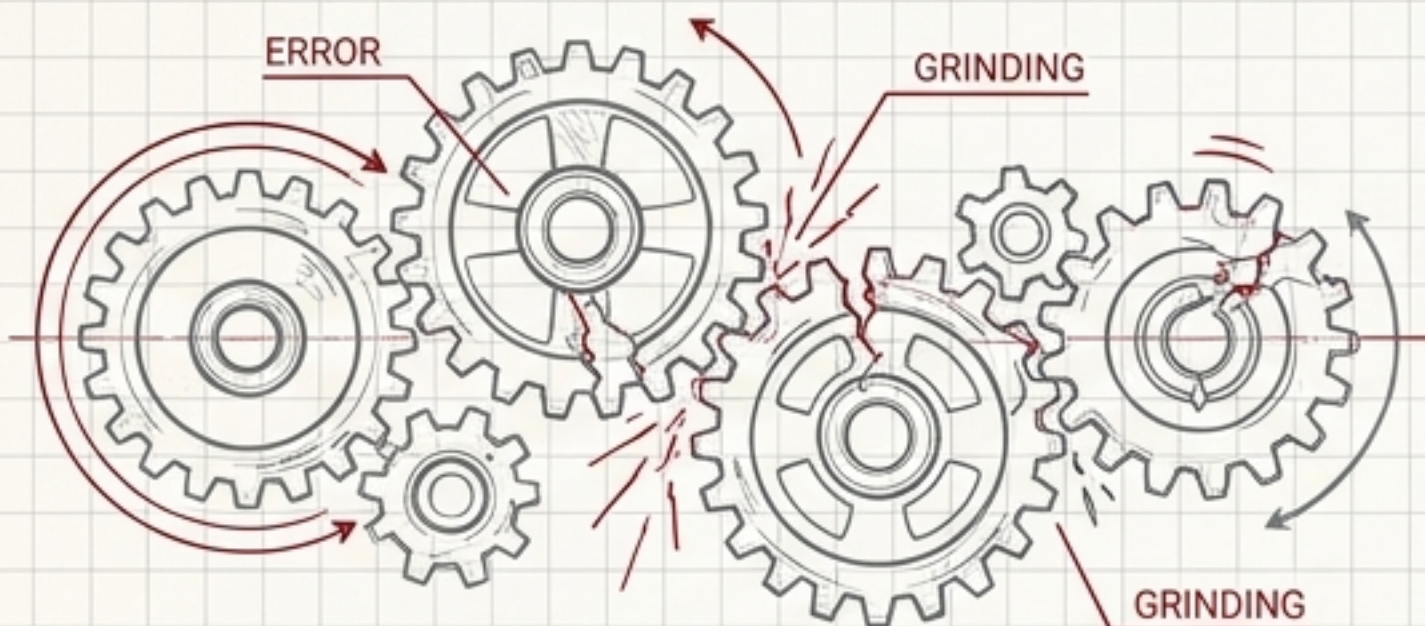
Engenharia Reversa: Dominando a mecânica oculta da Orientação a Objetos.



Sumário da Jornada



O Problema



```
q3 = q1.add(q2)
```

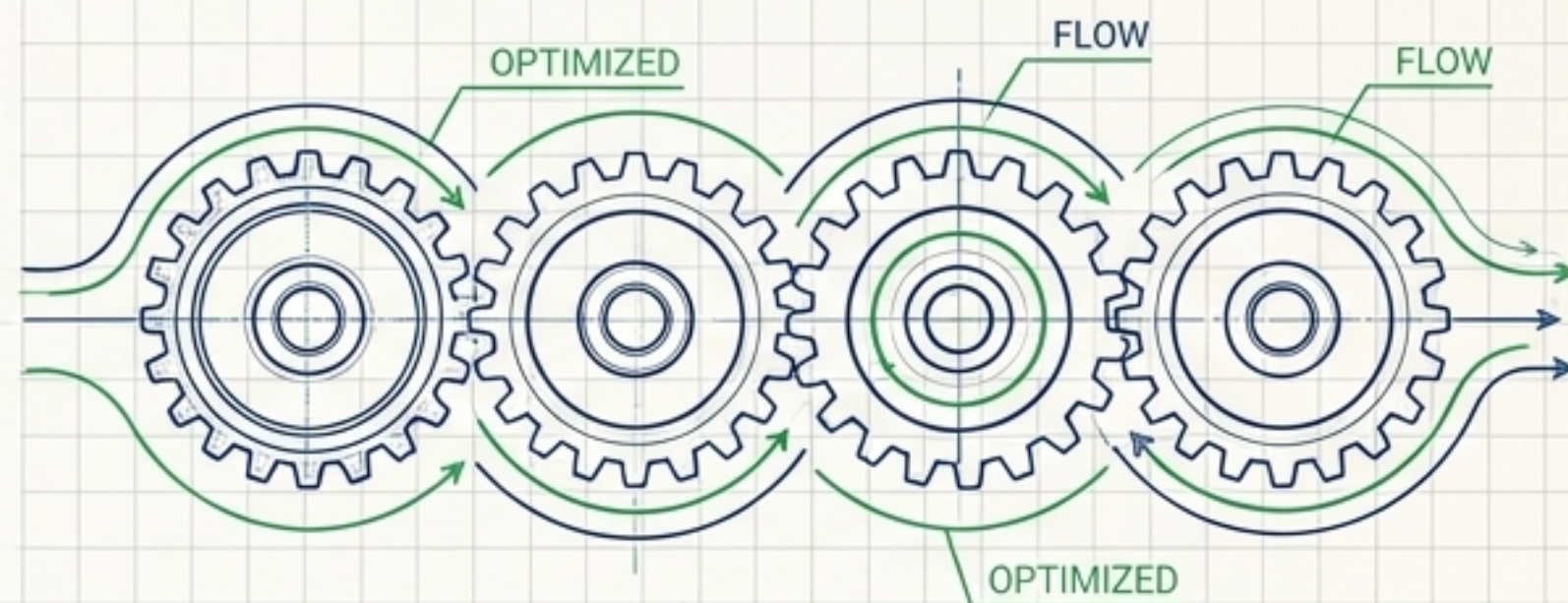
DIAGNOSTIC

Sintaxe verbosa, quebra a fluidez matemática.

SYSTEM NOTE

A Sobrecarga de Operadores (Operator Overloading) permite que classes personalizadas utilizem operadores nativos (+, -, *, ==), comportando-se exatamente como os tipos embutidos do Python.

A Solução



```
q3 = q1 + q2
```

DIAGNOSTIC

Código limpo, natural e altamente legível.

A rectangular box containing the mathematical expression $q1 + q2$. The box has a double border. Dimension lines indicate a width of 193mm and a height of 78mm. A dashed orange arrow points from the plus sign in the expression down to the code below.

O Python traduz o símbolo matemático diretamente para um método interno.

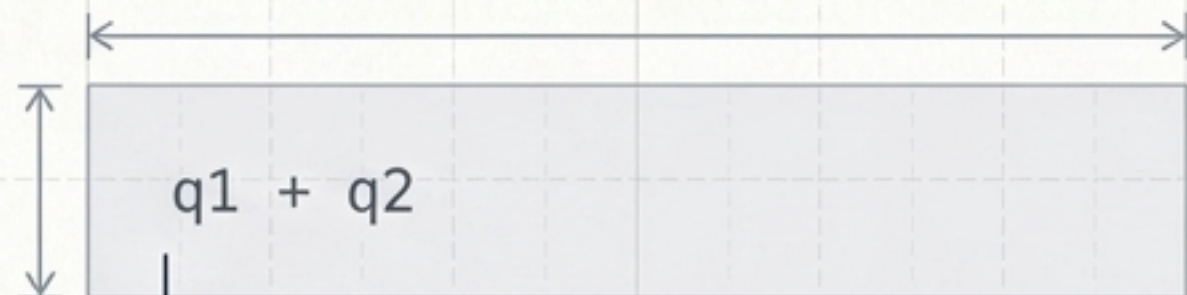
A diagram of a code editor window. The window has a title bar with three circles on the left. The code inside is `def __add__(self, other):`. The window has a double border. Dimension lines indicate a width of 150mm and a height of 78mm. A dashed orange arrow points from the plus sign in the expression above down to the code.

Esses métodos especiais começam e terminam com duplo sublinhado (double underscore), sendo conhecidos na comunidade como Dunder Methods.



Quando (NÃO) usar: O Princípio Semântico

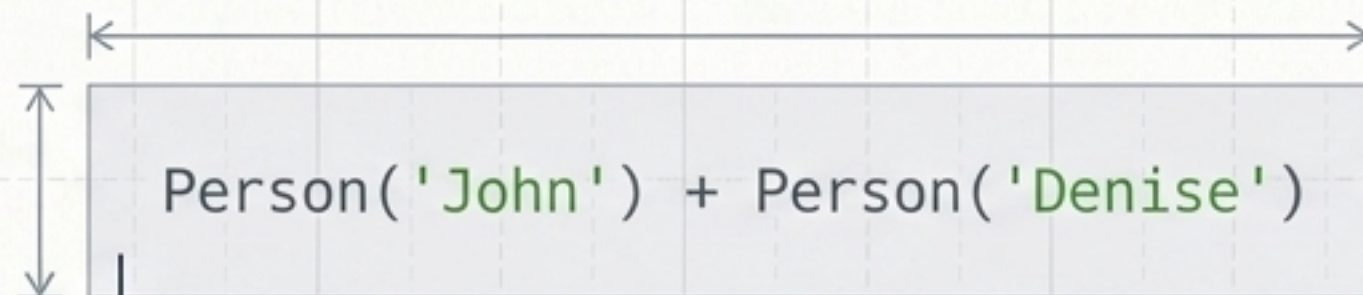
Domínio Matemático



Semântica clara. Representa a soma de dois valores.



Domínio de Negócios



Ambíguo. Eles estão se casando? Formando uma equipe? Qual é o objeto resultante?



A Regra de Ouro do Design: Siga a semântica dos tipos embutidos. Implemente apenas se fizer sentido matemático ou lógico indiscutível.  NotebookLM

Construindo a Base: A Classe Quantity

```
class Quantity:
```

```
    def __init__(self, value=0):  
        self.value = value
```

Encapsula o valor numérico.

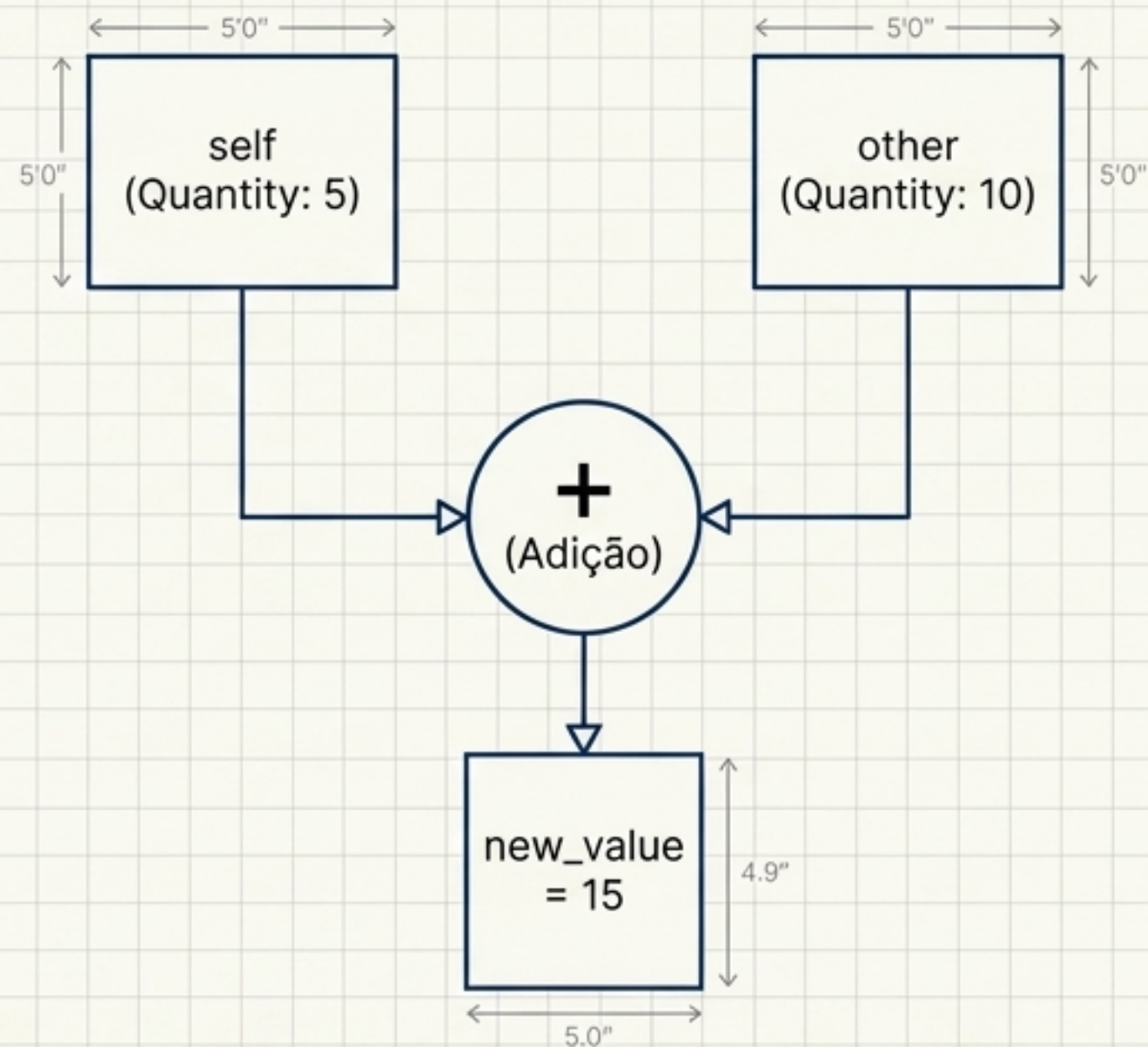
```
    def __str__(self):  
        return 'Quantity[' + str(self.value) + ']'
```

Garante legibilidade no print:
retorna 'Quantity[' + str(self.value) + ']'.

Antes de aplicar operadores matemáticos, nossa classe precisa de um construtor sólido e de uma representação de string formatada para facilitar o debug.

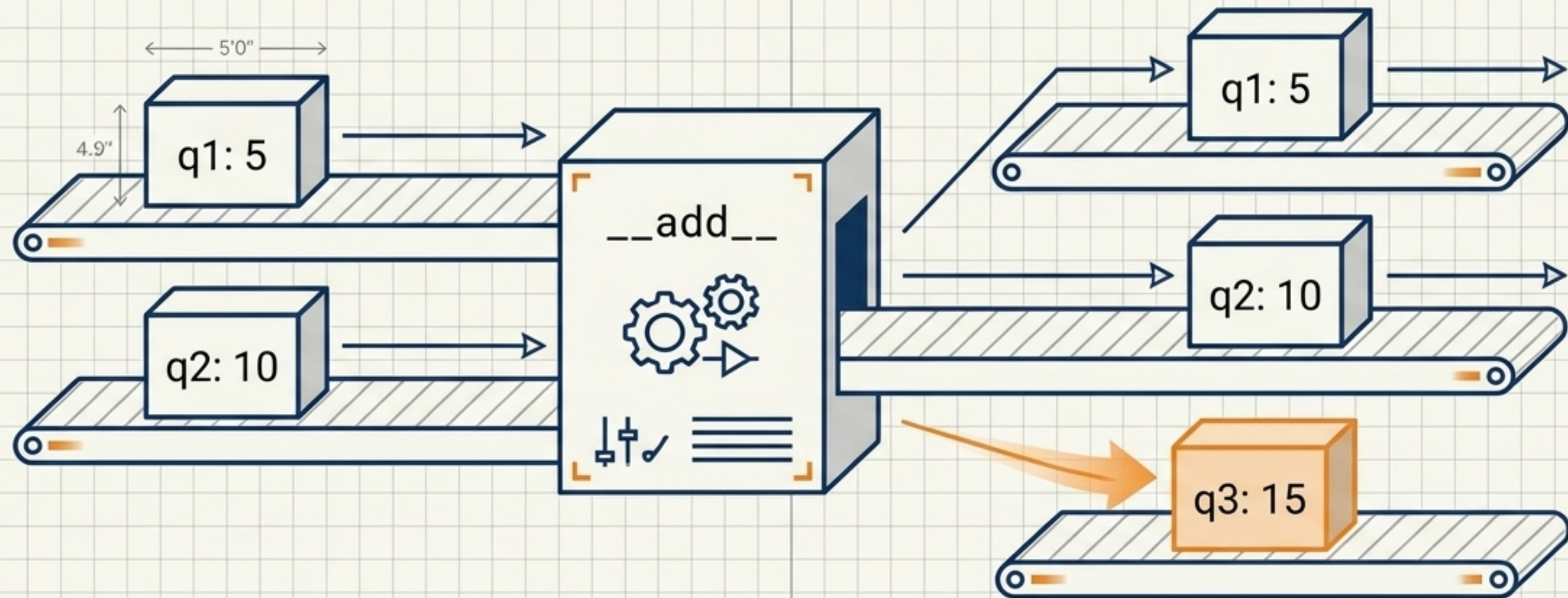
Implementando Adição e Subtração

```
def __add__(self, other):  
    new_value = self.value + other.value  
    return Quantity(new_value)  
  
def __sub__(self, other):  
    new_value = self.value - other.value  
    return Quantity(new_value)
```



A classe extrai o atributo numérico de si mesma (`self.value`) e do objeto secundário (`other.value`), executa a matemática nativa e encapsula o resultado.

O Padrão de Projeto: Imutabilidade



Assim como números inteiros no Python ($2 + 3 = 5$, onde nem o 2 nem o 3 sofrem alteração), objetos numéricos devem **ser imutáveis**. A operação sempre gera uma nova instância em vez de corromper os objetos originais.

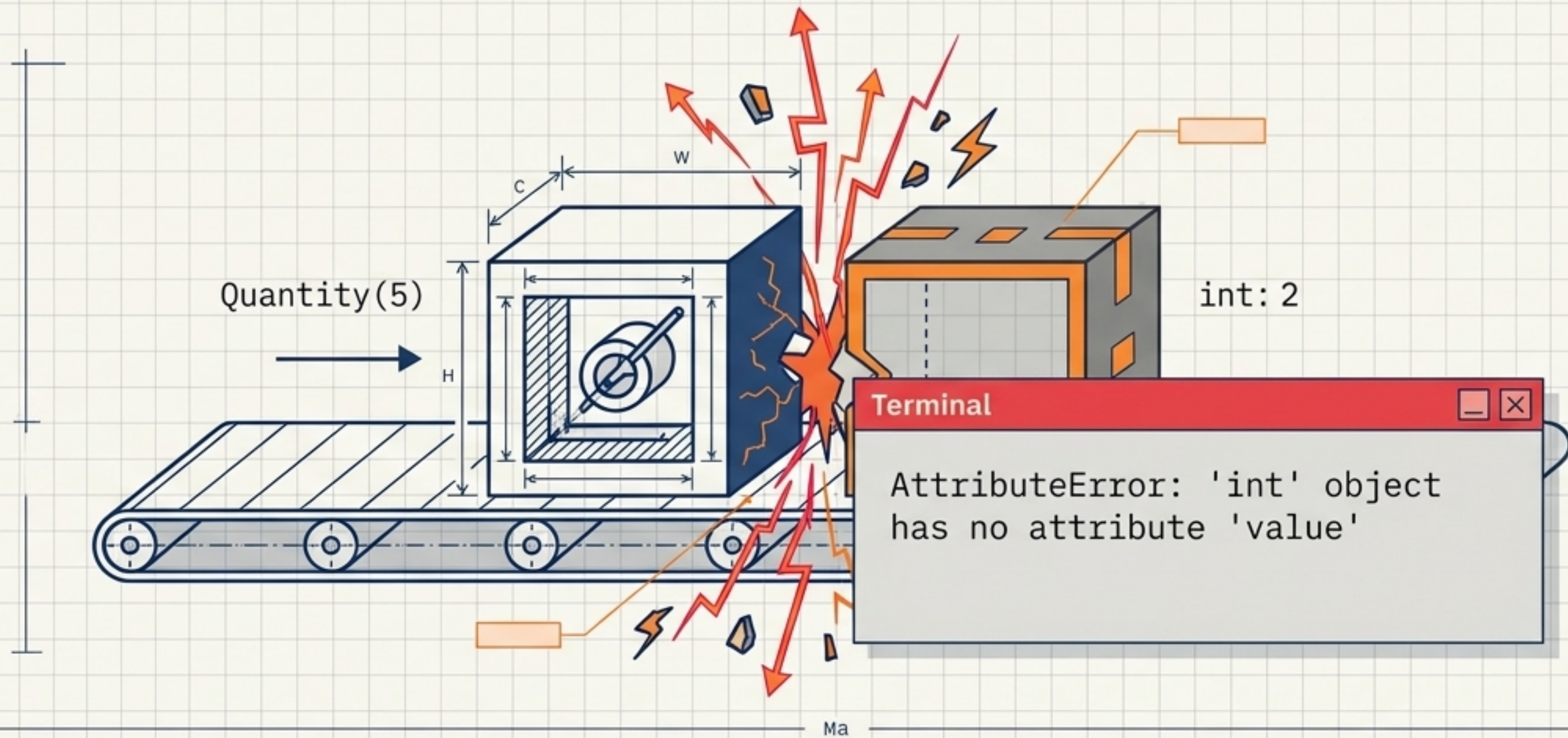
O Arsenal Matemático

Operação	Símbolo	Método Dunder
Multiplicação	*	<code>__mul__</code>
Potência	**	<code>__pow__</code>
Divisão	/	<code>__truediv__</code>
Divisão Inteira	//	<code>__floordiv__</code>
Módulo	%	<code>__mod__</code>

Warning Box

Nota de Design: O Python suporta operadores **bitwise** (<<, >>), mas os omitimos intencionalmente por não aderirem à semântica da nossa classe Quantity.

O Problema da Assimetria de Tipos



Até agora, nosso método assume que o lado direito da equação (other) sempre será outro objeto `Quantity`. Multiplicar por um inteiro nativo quebra o código, pois inteiros não possuem o atributo `.value`.

A Solução de Tipagem: isinstance()

```
def __mul__(self, other):
```

`isinstance(other, int)?`

Sim

Não

```
new_value = self.value * other
```

```
new_value = self.value * other.value
```

```
return Quantity(new_value)
```

O uso de `isinstance()` dota a classe de flexibilidade inteligente, permitindo que ela adapte sua lógica dependendo do tipo de dado recebido no argumento.

Operadores de Comparação

Operador	Expressão (Exemplo)	Método Dunder
<	q1 < q2	<code>__lt__</code>
<=	q1 <= q2	<code>__le__</code>
==	q1 == q2	<code>__eq__</code>
!=	q1 != q2	<code>__ne__</code>
>	q1 > q2	<code>__gt__</code>
>=	q1 >= q2	<code>__ge__</code>

Sobrecarregar operadores de comparação permite que nossas instâncias sejam ordenadas e avaliadas diretamente em blocos `if / else`.

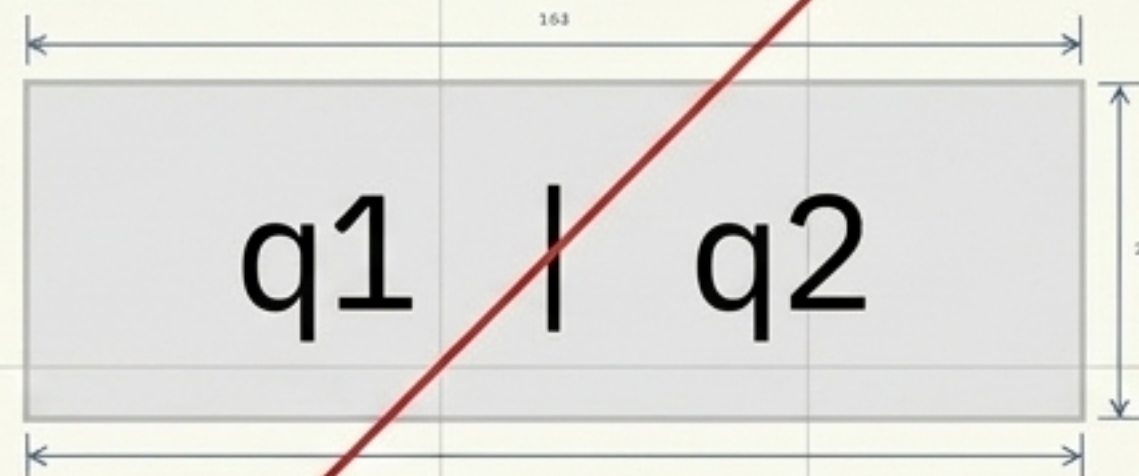
Diferente da matemática, estes métodos retornam valores estritos de `True` ou `False`.

Limites Lógicos: O Que Fica de Fora?

&

|

^



Operadores Lógicos (`__and__`, `__or__`, `__xor__`) também podem ser sobrecarregados. Contudo, retornamos à nossa Regra de Ouro: 'De que forma a Quantidade 1 é uma alternativa à Quantidade 2?'. Como não faz sentido semântico, nós não os implementamos.

Síntese Prática: O Exercício 'Distance'

Requisitos

- ✓ Math Methods
- ✓ String Formatting
- ✓ Type Checking (int)

O Desafio "Distance"

```
d1 = Distance(6)
d2 = Distance(3)
```

Output

```
Distance[9]
Distance[3]
Distance[3.0]
Distance[1]
Distance[6]
```

O teste final. Construir uma classe Distance que suporte adição, subtração e divisões flexíveis por inteiros consolida todas as engrenagens exploradas nesta jornada.

Resumo Executivo



1. Dunder Methods: São a ponte mecânica. O Python traduz símbolos de operadores nativos para chamadas de métodos especiais ocultos.



2. Imutabilidade Numérica: Operações em objetos que representam valores não devem alterar seu estado original, mas sim retornar novas instâncias.



3. Design Semântico: Implemente apenas operadores que possuam lógica indiscutível para o domínio da sua classe. Evite abusos sintáticos.



4. Flexibilidade e Segurança: Utilize testes condicionais com `isinstance()` para garantir que sua classe saiba interagir perfeitamente com os tipos nativos do Python.